

Deep Learning

Redes Recorrentes

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
3. Recorrência em Redes Neurais
4. Backpropagation Through Time
5. Considerações Práticas
6. Referências

Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
3. Recorrência em Redes Neurais
4. Backpropagation Through Time
5. Considerações Práticas
6. Referências

Sequências em Aprendizado de Máquina

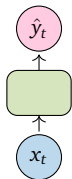
- Muitos problemas relevantes envolvem dados **sequenciais**, em que a ordem dos elementos importa.
- Exemplos:
 - Análise de sentimento de uma frase ou resenha.
 - *Image captioning*: gerar uma descrição textual a partir de uma imagem.
 - Tradução automática entre idiomas.
 - Previsão de séries temporais (preço de ações, clima, energia).
 - Reconhecimento de fala e síntese de áudio.
- Em todos esses casos, mudar a ordem dos elementos altera o significado da entrada ou da saída.

Análise de Sentimento

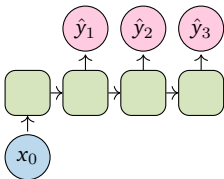
- A tarefa: dada uma frase, prever se o sentimento é positivo ou negativo.
- Frase 1: *“esse filme é uma bela porcaria”* $\Rightarrow$ sentimento **negativo**.
- Frase 2: *“eu esperava que fosse uma porcaria, mas é uma bela forma de passar o tempo”* $\Rightarrow$ sentimento **positivo**.
- As mesmas palavras (“bela”, “porcaria”) aparecem nas duas frases.
 - A **ordem** e o **contexto** são fundamentais para a classificação.

Tipos de Problemas Sequenciais

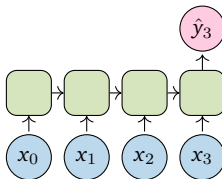
one-to-one



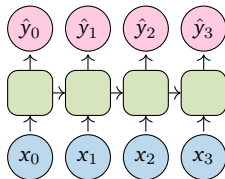
one-to-many



many-to-one



many-to-many



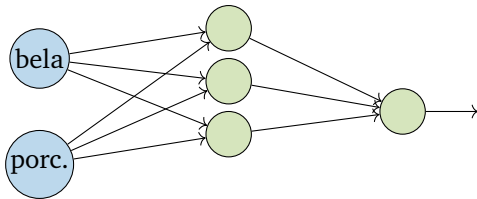
- *one-to-many*: image captioning (uma imagem gera uma sequência de palavras).
- *many-to-one*: análise de sentimento (uma sequência gera um rótulo).
- *many-to-many*: tradução automática, geração de texto.

Overview

1. Sequências
- 2. Tentando Modelar com MLP e CNN**
3. Recorrência em Redes Neurais
4. Backpropagation Through Time
5. Considerações Práticas
6. Referências

Uma Ideia: Usar um MLP

- Vamos tentar usar um MLP para classificar o sentimento de uma frase.



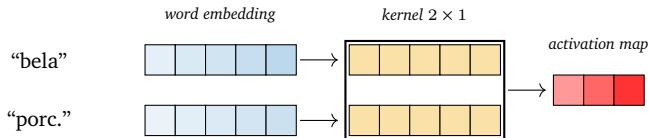
- Cada palavra vira um vetor de entrada (uma *embedding*).
- O MLP combina todas as entradas em uma única saída.

Problemas com MLP em Sequências

- **Frases de tamanhos diferentes** demandariam entradas de tamanhos diferentes, com pesos diferentes.
- Entradas maiores demandam mais pesos.
- A mesma palavra em posições diferentes é multiplicada por pesos diferentes.
 - Cada posição tem que aprender separadamente o significado da palavra.
- Não há mecanismo natural para tratar entradas de comprimento variável.

Outra Ideia: Usar uma CNN 1D

- Vamos tentar usar uma CNN para classificar o sentimento da frase.
- Cada palavra é convertida em uma *word embedding*, e um *kernel* desliza ao longo da sequência.



Limitações da CNN 1D

- Para frases mais longas, o problema fica mais nítido:
 - Contextos “distantes” são difíceis de modelar.
 - Precisamos empilhar várias camadas convolucionais para aumentar o campo receptivo.
- Mesmo com várias camadas, o campo receptivo é finito e o tamanho da saída depende do tamanho da entrada.

Não são Boas Soluções

- MLPs exigem entradas de tamanho fixo. Frases têm tamanho variável.
- CNNs podem trabalhar com entradas de tamanho variável, mas se não tomarmos cuidado com o campo receptivo corremos o risco de não levar em conta a frase toda.

Precisamos de uma rede que:

- Trabalhe sequencialmente com as palavras, uma por vez.
- Mantenha um **estado** (memória) que carregue informações obtidas anteriormente.

Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
- 3. Recorrência em Redes Neurais**
4. Backpropagation Through Time
5. Considerações Práticas
6. Referências

Revisitando o MLP

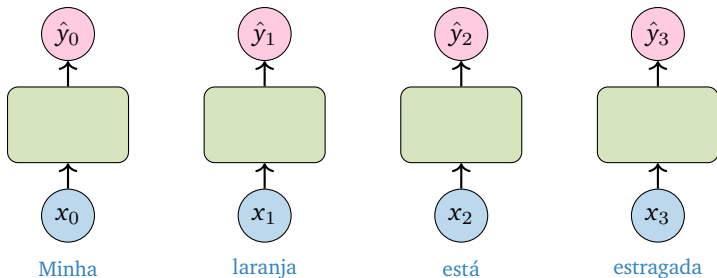
- Em uma camada densa, temos:
 - entradas x ,
 - pesos θ ,
 - função de ativação,
 - saídas $\hat{y}$.
- Podemos colapsar todo o vetor de entrada em um único nó e a camada inteira em um único bloco.



- O vetor de entrada foi **colapsado**, mas continua tendo um tamanho fixo.

Aplicando o MLP a uma Sequência

- Vamos rodar esse MLP na frase “*Minha laranja está estragada*”, palavra a palavra.



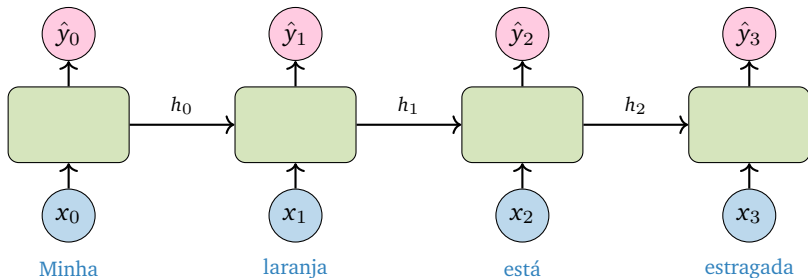
- Cada palavra é processada de forma independente.
- O modelo **não sabe** que “laranja” veio depois de “Minha”.

O Problema Continua

- Qual é o problema dessa abordagem?
 - O **processamento de cada entrada é individual** e não leva em conta o restante da sequência.
 - Não há **memória** entre os instantes.
- Para um problema sequencial, precisamos que a saída em t dependa também do que aconteceu em $t - 1, t - 2, \dots$

Solução: Recorrência

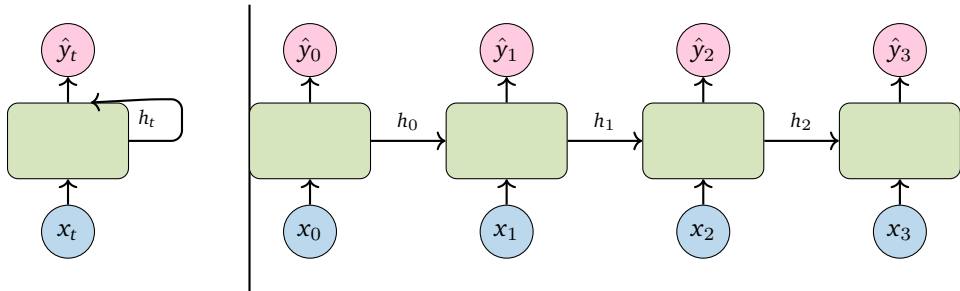
- Além da entrada x_t no instante atual, vamos passar adiante alguma informação do instante anterior.
- Chamamos essa informação de **estado interno** h_t .



- Os mesmos pesos são reutilizados em todos os passos de tempo.

Versão Dobrada vs Desdobrada

- A representação **desdobrada no tempo** pode ser compactada em uma versão **dobrada**, com uma única célula que se conecta a si mesma.



- Os dois desenhos representam a **mesma rede**: a versão desdobrada apenas explicita os passos de tempo.

Camadas Densas com Recorrência

- A saída de uma célula recorrente depende da **entrada** e do **estado interno**.
- Os pesos são reutilizados em todos os passos de tempo.

$$\hat{y}_t = f(\underbrace{x_t}_{\text{entrada}}, \underbrace{h_{t-1}}_{\text{memória}})$$

- O estado interno h_{t-1} resume tudo o que a rede “viu” até o passo anterior.

Pseudocódigo de uma RNN

```
1 rnn = RNN()
2 hidden_state = [0, 0, 0, 0]
3
4 sentence = ["Minha", "laranja", "está"]
5
6 for word in sentence:
7     prediction, hidden_state = rnn(word, hidden_state)
8
9 next_word = prediction
10 print(next_word)
11 # >>> "estragada"
```

- A cada iteração, a rede recebe a entrada atual e o estado interno, e devolve a predição e o novo estado.

Detalhando as Atualizações

- A célula recorrente mais simples (*vanilla RNN*) é dada por:

$$h_t = \tanh(\theta_{hh}^T h_{t-1} + \theta_{xh}^T x_t + b_h)$$

$$\hat{y}_t = \theta_{hy}^T h_t + b_y$$

- θ_{xh} : pesos da entrada para o estado interno.
- θ_{hh} : pesos do estado interno anterior para o novo estado interno.
- θ_{hy} : pesos do estado interno para a saída.
- b_h, b_y : *bias*.
- Em *frameworks* como PyTorch é comum encontrar a notação $W_h h_{t-1} + W_x x_t + b$. Aqui usamos θ_{hh} e θ_{xh} para sublinhar a distinção entre as conexões *hidden-to-hidden* e *input-to-hidden*.

Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
3. Recorrência em Redes Neurais
- 4. Backpropagation Through Time**
5. Considerações Práticas
6. Referências

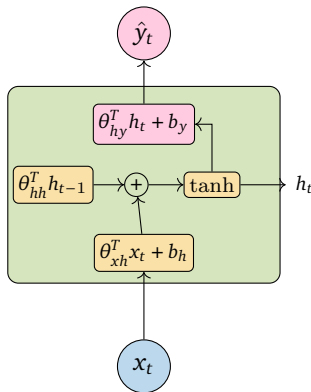
Backpropagation Through Time (BPTT)

- O algoritmo de treinamento é o mesmo já conhecido: *backpropagation*.
- Porém agora precisamos considerar múltiplos passos de tempo (*timesteps*).
 - O mesmo peso influencia a saída de várias formas diferentes, conforme o *timestep*.
 - As contribuições do gradiente são **somadas** ao longo da sequência.
- Esse algoritmo é chamado de *Backpropagation Through Time*.

Setup do Exemplo

Vamos usar a rede recorrente mais simples possível para um exemplo numérico:

- $x = [1, 2]$
- $y = 3$
- $h_0 = 0$
- $\theta_{xh} = 0,5, \quad \theta_{hh} = 0,5$
- $\theta_{hy} = 1$
- $b_h = 0, \quad b_y = 0$

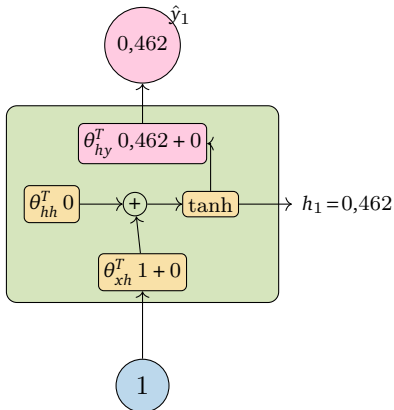


Forward Pass, Passo 1

- No passo 1, $x_1 = 1$ e $h_0 = 0$:

$$h_1 = \tanh(\theta_{xh} \cdot 1 + \theta_{hh} \cdot 0 + b_h) = \tanh(0,5) \approx 0,462$$

$$\hat{y}_1 = \theta_{hy} \cdot h_1 + b_y = 1 \cdot 0,462 + 0 = 0,462$$

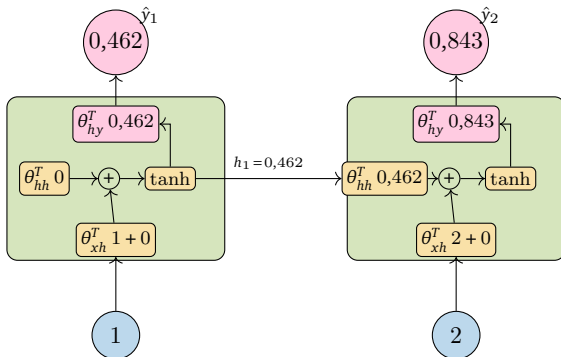


Forward Pass, Passo 2

- No passo 2, $x_2 = 2$ e $h_1 \approx 0,462$:

$$h_2 = \tanh(\theta_{xh} \cdot 2 + \theta_{hh} \cdot 0,462 + b_h) = \tanh(1,231) \approx 0,843$$

$$\hat{y}_2 = \theta_{hy} \cdot h_2 + b_y = 1 \cdot 0,843 + 0 = 0,843$$



Cálculo da *Loss*

- Após o *forward*, comparamos $\hat{y}_2$ com a meta $y = 3$ usando o erro quadrático médio:

$$J(\theta) = \frac{1}{2}(\hat{y}_2 - y)^2 = \frac{1}{2}(0,843 - 3)^2 \approx 2,326$$

- A *loss* no instante final pode ser usada sozinha (*many-to-one*) ou somada ao longo de todos os instantes (*many-to-many*).

Backward Pass: Pesos da Saída

- Vamos calcular o gradiente de J em relação aos parâmetros, começando pelos pesos da saída.

$$\begin{aligned}\frac{\partial J}{\partial \theta_{hy}} &= \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \theta_{hy}} = (\hat{y} - y) h_2 \\ &= (0,843 - 3) \cdot 0,843 \approx -1,818\end{aligned}$$

$$\begin{aligned}\frac{\partial J}{\partial b_y} &= \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_y} = (\hat{y} - y) \cdot 1 \\ &= (0,843 - 3) \cdot 1 \approx -2,157\end{aligned}$$

- As derivadas de $\hat{y}_2$ em relação aos pesos da saída têm a mesma forma do MLP padrão.

Backward Pass: Pesos da Entrada

- Para os pesos θ_{xh} , o gradiente atravessa todos os passos de tempo:

$$\frac{\partial J}{\partial \theta_{xh}} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial z_2} \left(\frac{\partial z_2}{\partial \theta_{xh}} + \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial \theta_{xh}} \right)$$

- Substituindo as expressões da *vanilla RNN*:

$$\frac{\partial J}{\partial \theta_{xh}} = (\hat{y} - y) \theta_{hy} (1 - \tanh^2(z_2)) \left(x_2 + \theta_{hh} (1 - \tanh^2(z_1)) x_1 \right)$$

- Numericamente:

$$\frac{\partial J}{\partial \theta_{xh}} = (0,843 - 3) \cdot 1 \cdot (1 - 0,843^2) \left(2 + 0,5(1 - 0,462^2) \cdot 1 \right) \approx -1,493$$

Backward Pass: *bias* e Pesos Recorrentes

- Aplicando a mesma ideia ao *bias* b_h e aos pesos recorrentes θ_{hh} :

$$\begin{aligned}\frac{\partial J}{\partial b_h} &= (\hat{y} - y) \theta_{hy} (1 - \tanh^2(z_2)) (1 + \theta_{hh} (1 - \tanh^2(z_1))) \\ &\approx (0,843 - 3) \cdot 1 \cdot (1 - 0,843^2) (1 + 0,5(1 - 0,462^2)) \approx -0,870\end{aligned}$$

$$\begin{aligned}\frac{\partial J}{\partial \theta_{hh}} &= (\hat{y} - y) \theta_{hy} (1 - \tanh^2(z_2)) (h_1 + \theta_{hh} (1 - \tanh^2(z_1)) h_0) \\ &\approx (0,843 - 3) \cdot 1 \cdot (1 - 0,843^2) (0,462 + 0,5(1 - 0,462^2) \cdot 0) \approx -0,288\end{aligned}$$

- Como $h_0 = 0$, o segundo termo de $\partial J / \partial \theta_{hh}$ se anula.

Resumo dos Gradientes

Parâmetro	$\partial J / \partial \cdot$
θ_{hy}	-1,818
b_y	-2,157
θ_{xh}	-1,493
b_h	-0,870
θ_{hh}	-0,288

- Repare que o gradiente do peso recorrente θ_{hh} depende dos estados internos passados.
- Em sequências longas, esse caminho de derivadas multiplicadas é a origem dos problemas de *vanishing* e *exploding gradient*.

Passo do *Optimizer*

- Com taxa de aprendizado $\eta = 0,1$ e os gradientes calculados:

$$\theta_{hy} \leftarrow \theta_{hy} - \eta \frac{\partial J}{\partial \theta_{hy}} = 1 - 0,1 \cdot (-1,818) = 1,1818$$

$$b_y \leftarrow 0 - 0,1 \cdot (-2,157) = 0,2157$$

$$\theta_{xh} \leftarrow 0,5 - 0,1 \cdot (-1,493) = 0,6493$$

$$b_h \leftarrow 0 - 0,1 \cdot (-0,870) = 0,087$$

$$\theta_{hh} \leftarrow 0,5 - 0,1 \cdot (-0,288) = 0,5288$$

- Refazendo o *forward* com os pesos atualizados, $\hat{y}_2 \approx 1,324$ e $J \approx 1,404$, ou seja, a *loss* caiu de 2,326 para 1,404.

Implementação em PyTorch

```
1 class RNNsImples(torch.nn.Module):
2     def __init__(self) -> None:
3         super().__init__()
4         self.theta_hh = torch.nn.Parameter(torch.tensor(0.5))
5         self.bias_h    = torch.nn.Parameter(torch.tensor(0.0))
6         self.theta_xh = torch.nn.Parameter(torch.tensor(0.5))
7         self.theta_hy = torch.nn.Parameter(torch.tensor(1.0))
8         self.bias_y    = torch.nn.Parameter(torch.tensor(0.0))
9
10    def forward(self, x, h=torch.tensor(0.0)):
11        h = torch.tanh(x * self.theta_xh + h * self.theta_hh + self.bias_h)
12        y = h * self.theta_hy + self.bias_y
13        return y, h
```

Loop de Treinamento

```
1 model = RNNSimples()
2 optimizer = SGD(model.parameters(), lr=0.1)
3 x = [torch.tensor(1.0), torch.tensor(2.0)]
4 y = torch.tensor(3.0)
5
6 model.train()
7 optimizer.zero_grad()
8 h = torch.tensor(0.0)
9 for x_i in x:
10     y_hat, h = model(x_i, h)
11     loss = (y - y_hat)**2 / 2
12     loss.backward()
13     optimizer.step()
```

- O `loss.backward()` acumula o gradiente de cada peso ao longo de todos os passos de tempo automaticamente.

Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
3. Recorrência em Redes Neurais
4. Backpropagation Through Time
- 5. Considerações Práticas**
6. Referências

Problemas: *Vanishing* e *Exploding Gradient*

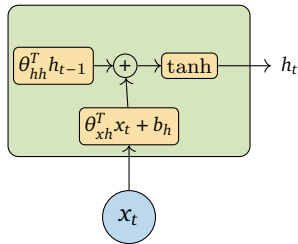
- Os velhos conhecidos: *vanishing* e *exploding gradient*.
- Com RNNs ficam piores, pois a rede também é **profunda no tempo**.
 - Em sequências longas, o gradiente se multiplica por $\theta_{hh} (1 - \tanh^2)$ a cada passo.
 - Após T passos, o produto de Jacobianos toma a forma:

$$\prod_{k=1}^T \theta_{hh} (1 - \tanh^2(z_k))$$

- Se cada fator é menor que 1, o produto decai exponencialmente em T (*vanishing*); se é maior que 1, cresce exponencialmente (*exploding*).
- Soluções comuns:
 - *Gradient clipping by norm*, com limiar c : se $\|g\| > c$, então $g \leftarrow c \cdot g / \|g\|$. Preserva a direção do gradiente. Variantes *clipping by value* distorcem a direção e são menos usadas.
 - Arquiteturas com portas: LSTM e GRU (próxima aula).

Uma Observação sobre PyTorch

- É comum que *frameworks* como o PyTorch **não** incluam a camada de saída $\hat{y}$ diretamente no bloco RNN.
- Cabe ao desenvolvedor adicionar uma camada densa (`nn.Linear`) sobre o estado interno h_t .
- Isso dá flexibilidade para combinar a célula recorrente com diferentes camadas de saída (regressão, classificação, sequência).



Overview

1. Sequências
2. Tentando Modelar com MLP e CNN
3. Recorrência em Redes Neurais
4. Backpropagation Through Time
5. Considerações Práticas
- 6. Referências**

Referências I

- Sugere-se a leitura do Capítulo 10 de *Deep Learning*¹.
 - Disponível em <https://www.deeplearningbook.org/>.
- Material baseado em:
 - Capítulo 10 de *Deep Learning*, Ian Goodfellow, Yoshua Bengio e Aaron Courville.
 - *MIT Introduction to Deep Learning*, Ava Amini, 2023.

[1] Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.

¹Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.